

MedESQ: Rethinking Efficient Text-to-NoSQL Querying Generation with a Complexity Taxonomy for Lucene Data Structures

Anonymous ACL submission

Abstract

NoSQL databases offer distinct advantages, such as flexibility in managing unstructured data and efficient traversal of complex user intents. These characteristics make the text-to-NoSQL methods particularly promising for question answering and knowledge screening in healthcare, where medical records and reports are often heterogeneous and semi-structured. Although extensive research has been conducted in the field of query generation for natural language querying, a significant gap remains in translating these advancements into real applications, particularly in query efficiency. To address this gap, we formally define a complexity protocol for NoSQL queries using Elasticsearch (ES) as a representative backend, and construct a complexity-aware **Medical text-to-ElasticSearch Queries (MedESQ)** dataset derived from the Vaccine Adverse Event Reporting System (VAERS) data. Building on this foundation, we further propose a two-stage framework, **Retrieval-Augmented pipeline for Complexity-aware text-to-ESQ generation (RACE-ESQ)**, that integrates RAG to explicitly optimize query complexity during generation and is evaluated on the new **MedESQ** data. Experimental results show that **RACE-ESQ** improves both generation quality and query efficiency.

1 Introduction

Natural language querying (NLQ) helps convert human intent to different kinds of structured queries (Deng et al., 2021), which plays a pivotal role in enhancing the accessibility of purpose-built databases, thus extending their utility to a broader audience. Despite the growing popularity of text-to-SQL (Deng et al., 2022), current NLQ datasets offer limited support for NoSQL scenarios, while in many practical environments, NoSQL also plays an important role. For example, Elasticsearch is widely used to integrate heterogeneous medical records

Why Spider/BIRD Complexity Rules Break

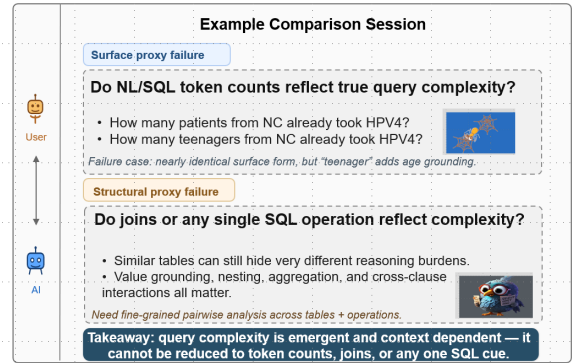


Figure 1: An example for taxonomy in different datasets

in many web-based applications. Further, NoSQL databases are crucial in various industrial settings and emerging applications due to their efficient data models and schema flexibility (Lu et al., 2025; Qin et al., 2025; Hossen et al., 2023).

Although extensive research has been conducted in the field of query generation, a significant gap remains in translating these advancements into real applications, specifically concerning query efficiency. However, in a real search system, correctness alone is not enough. A query can be semantically right and still be operationally poor if it uses an unnecessarily expensive operator. Such discrepancies in execution time directly affect the overall server’s capability and represent a critical factor in determining the operational cost of web-based applications (Ramu, 2023). Existing NLQ datasets are highly dependent on their training data distributions and often do not reflect the true computational behavior of queries, like the Spider and BIRD datasets in Figure 1. Moreover, the exploration of query efficiency in the NoSQL domain remains largely unaddressed.

Some initial pipeline has focused on SQL query efficiency, exemplified by EllieSQL (Zhu et al., 2025), focus mainly on high-level techniques such

as prompt optimization or post-training adjustments. While these strategies provide incremental improvements, the absence of deeper fundamental analysis leads to a substantial dependence on the LLM’s intrinsic performance. On the other hand, SQL’s functional constraints, especially its limited support for full-text retrieval and heterogeneous information types contribute to the suboptimal results observed in these settings (Pourreza et al., 2025; Zhang et al., 2024), which motivated the focus on NoSQL solutions. Compared to the challenges inherent to SQL, the novel database structure and heterogeneous operations within NoSQL query pipelines necessitate more fine-grained analysis to enhance overall execution efficiency. In this work, we analyze Elasticsearch, one of the most widely used Lucene-based systems. Despite their pervasive use, there is currently no systematic analysis of these underlying structures. In terms of operational capabilities, Elasticsearch also exposes a substantially richer set of query operators covering text search, boolean composition, relevance scoring, and making its query behavior both more expressive in representing human intents and more complex than traditional relational systems.

Effectively addressing these challenges requires a more thorough fundamental analysis of query behavior, complemented by high-quality and accessible datasets and a framework to apply the cost-saving strategy guides in the LLM-based system. Our main contributions are summarized as follows:

- **A formal design of Elasticsearch query complexity taxonomy.** To the best of our knowledge, this represents the first systematic, hierarchical complexity analysis for Lucene data structures and NoSQL databases.
- **A comprehensive dataset, namely MedESQ, for text-to-ESQ in healthcare with consideration of query complexity.** Based on our analysis, we create the first systematic, hierarchical complexity-aware database for the text-to-NoSQL task in the healthcare domain.
- **A framework for efficiency-oriented text-to-ESQ.** To assess the effectiveness of our benchmark, we propose a tailored two-stage framework **RACE-ESQ**, which leverages Retrieval-Augmented Generation (Lewis et al., 2020) and **MedESQ**, and we conduct extensive experiments using several state-of-the-art LLMs.

2 Related Work

2.1 Natural Language Querying (NLQ)

NLQ is a foundational task in natural language processing that empowers users to interact with databases through everyday language rather than formal query syntax. Early research in NLQ has predominantly targeted text-to-SQL, where established SQL benchmarks, such as Spider (Yu et al., 2018), have targeted complex and cross-domain problems. By contrast, the NoSQL domain remains in its infancy, with only a few initial datasets like VAERSESQ (Zhang et al., 2023). Given NoSQL’s critical role in large-scale data management, it represents a fertile area for further study.

2.2 Efficiency-Oriented Query Generation

When queries are generated automatically, higher efficiency is required in the query generation and execution process to meet practical industrial demands. Early text-to-SQL work emphasizes syntactic correctness and execution accuracy, with benchmarks such as Spider (Yu et al., 2018) and Spider 2.0 (Lei et al., 2024) that treat queries as symbolic programs, categorizing queries through structural complexity. Current efficiency-oriented approaches, such as EllieSQL (Zhu et al., 2025) and Gen-SQL (Shi et al., 2025), adopt high-level strategies including prompt optimization or post-training refinements. In parallel, existing NoSQL datasets, which contributed to many important fields but remain largely underexplored in this field, highlight the need for precision efficiency analysis of NoSQL queries for NLQ.

2.3 Lucene Data Structures

Lucene retrieval systems (Bialecki et al., 2012) are built on efficient data structures such as inverted indexes for text search, finite state transducers (FSTs) for compact term dictionaries, and block k-d (BKD) trees for numeric and range queries. These components enable scalable and expressive retrieval, which are widely deployed in large-scale production systems such as Wikipedia, LinkedIn, Twitter, and Netflix, but how to analyze the whole construction for specific applications remains challenging.

3 The Complexity Protocol of ESQ

In this section, we describe the design and organization of our complexity protocol for ESQ, which is also applicable to Lucene-based data structures and

Table 1: Notations used in this paper.

Symbol	Definition
N	Total # of documents in the index.
V	Vocabulary size (# of unique indexed terms).
$ t $	Byte length of term t .
$df(t)$	Document frequency of t .
T	Set of query terms.
M	# of dictionary terms matching a pattern.
W	# of arcs in the FST term dictionary.
R	# of documents satisfying the query.
K	Requested top- K results (size).
U	# of unique buckets in a terms aggregation.

Table 2: Time complexity classes.

Function Sample	Time Complexity
A. ID lookup	$O(1)$
B. Term	$O(t + df(t)) \approx O(df(t))$
C. Boolean	$O(\sum_{i=1}^k df(t_i))$
D. Range/Prefix	Numeric (BKD): $O(\log N + R)$; String: $O(\ell + M + \sum_{i=1}^M df(t_i))$
E. Wildcard/Regex	$O(W + M + \sum_{i=1}^M df(t_i))$
F. Full-text	$O(\sum_{i=1}^m df(t_i)) + O(R \log K)$
G. Aggregation/Sort	Base: $O(\sum df(t_i))$; Agg.: $O(R) + O(U \log B)$; Sort: $O(R \log K)$

other similar NoSQL databases. We also present the formal definition of the text-to-ESQ task.

3.1 Elasticsearch Query Time Complexity

We organize the seven-class time complexity taxonomy for ES queries and provide the formal logic, rules, and proof sketches. There’s a notation-first approach, then lists the classes, formal complexity expressions, short proofs (based on Lucene data structures), and concise examples suitable for reproduction (in the Appendix section).

Table 1 summarizes all notations used in this paper. All complexity statements below are expressed as asymptotic costs in terms of the dominant terms (FST traversal, postings enumeration, BKD traversal, doc-value scans). We assume standard Lucene implementations based on Apache Lucene documentation¹, where the term dictionary is an FST, postings lists are compressed sorted docID sequences, numeric/date/geo fields use BKD trees, and doc-values serve sorting/aggregation.

3.2 Proof Sketches of Query Complexity

Table 2 lists the seven classes with their dominant complexity expression. This section provides a brief proof sketch that explains why ES (Lucene)

exhibits the stated complexity for each class.

3.2.1 Class A: Constant Lookup

Description Direct document retrieval by identifier (GET <index>/_doc/<id>).

Complexity $O(1)$ for average, or hash lookup.

Proof Sketch A Elasticsearch maintains an in-memory mapping from external `_id` values to Lucene document pointers (segment identifier and document ordinal). A single hash-table lookup (average constant time) retrieves the pointer; fetching stored fields by pointer incurs an additional constant-time operation (I/O cost aside). No postings lists or term-dictionary traversal is required.

3.2.2 Class B: Single-Term

Description Exact-match term query on a keyword or analyzed token field (term).

Complexity $O(|t| + df(t)) \approx O(df(t))$.

Proof Sketch B Lucene’s term dictionary is implemented as an FST: finding the term requires following $|t|$ arcs ($O(|t|)$). Once located, the query obtains a pointer to the postings list and must enumerate the $df(t)$ postings (reading compressed docIDs or blocks). Enumeration cost is linear in $df(t)$, which dominates the short FST lookup for typical $|t|$. Hence the stated bound.

3.2.3 Class C: Boolean Composition

Description Conjunctions or disjunctions of many term clauses (Boolean queries with must/should/filter).

Complexity $O(\sum_{i=1}^k df(t_i))$ for k leaf clauses (worst-case to fetch and merge all postings).

Proof Sketch C Each leaf term requires an FST lookup followed by reading its postings list with complexity $O(df(t_i))$. Intersecting (AND) or unioning (OR) sorted postings lists can be implemented with linear merge algorithms, which is bounded by the sum of the lengths of the lists. Efficient intersection orders (smallest-first) reduce average cost but do not improve the worst-case linear bound in the sum of postings.

¹https://lucene.apache.org/core/9_1_0/index.html

3.2.4 Class D-1: Numeric BKD Ranges

Description Numeric block k-d (BKD) ranges (For example numeric/date/geo).

Complexity $O(\log N + R)$.

Proof Sketch D-1 Lucene stores numeric-like indexed values using BKD trees. Locating relevant leaf cells is a balanced-tree traversal with complexity $O(\log N)$; enumerating matching documents across the chosen cells costs $O(R)$.

3.2.5 Class D-2: String-based Ranges

Description String-based range and prefix (For example A/B/C).

Complexity $O(|\text{bound or prefix}| + M + \sum_{i=1}^M df(t_i))$, where M is the count of distinct terms in V within the range or under the prefix.

Proof Sketch D-2 Seek in the FST to the lower bound or prefix node, enumerate the M matching dictionary terms ($O(M)$), then fetch and merge the postings of those M terms ($O(\sum df)$).

3.2.6 Class E: Pattern Queries

Description Wildcard and regexp queries are compiled to finite automata and intersected with the FST to enumerate matching terms.

Complexity $O(W + M + \sum_{i=1}^M df(t_i))$ in the worst case, where W denotes FST arc visits.

Proof Sketch E The automaton derived from the pattern is intersected with the term-dictionary FST. This can produce a product automaton whose traversal may visit a large portion (or all) of the FST. Therefore, W is an upper bound on the traversal. Each matched term (M terms) and its postings are both enumerated. The postings-scan term dominates when matched terms have large values.

3.2.7 Class F: Full-text Queries

Description Analyzed text queries: the input string is tokenized (analyzer), producing tokens $t_1, \dots, t_m$, which are then queried as terms. Scoring (e.g., BM25) is accumulated per posting.

Complexity $O(\sum_{i=1}^m df(t_i)) + O(R \log K)$ (plus a negligible $O(L)$ analyzer cost), where K is the requested top- K results.

Proof Sketch F Tokenization is linear in the query length L . For each token, the engine performs an FST lookup and enumerates postings. As postings are visited, scores are accumulated in an in-memory accumulator (or heap for top- K). Maintaining a top- K heap over R candidates yields the $O(R \log K)$ complexity term. The postings enumeration cost is dominant in large document number settings.

3.2.8 Class G: Aggregations and Sorting

Description Bucketed aggregations (For example terms/histogram) and sorts require scanning doc-values for matched documents.

Complexity Base matching cost $O(\sum df(t_i))$; plus aggregation overhead $O(R)$ to scan doc-values and accumulate counts; selecting top- B buckets costs $O(U \log B)$, and sorting top- K results costs $O(R \log K)$.

Proof Sketch G After computing the set of matching docs, each matching document's doc-value is read (constant time) and used to update aggregation counts (hash table or ordinal array). Extracting the top buckets requires a selection step whose cost depends on the number of unique keys seen. Sorting similarly requires maintaining a size- K heap while scanning candidates.

3.3 Elasticsearch Query Difficulty Taxonomy

Building on the foundational complexity taxonomy of Elasticsearch functions, we group the Elasticsearch queries into three difficulty tiers. The Easy category includes classes A, B, and C. The Medium category includes classes D and E, and the Hard category includes classes F and G based on their sensitivity to dataset size. As data volumes increase, the complexity gap between these tiers widens significantly. In general, combining functions within the same tier maintains that tier's characteristic complexity. However, the inclusion of even a single function from a higher tier immediately elevates the query's overall cost to that higher level.

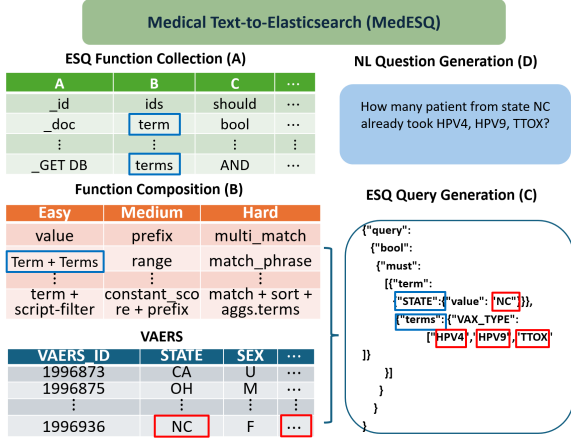


Figure 2: An example in the MedESQ dataset. Colors boxed show the correspondence between various components in the Basic Function, Function Composition, the NL question, and the ES query.

3.4 Formal Definition of Text-to-ESQ

We define *text-to-ESQ* as the task of translating a natural language query into an executable ES query. Let x be the input, y the target, and $\hat{y}$ the predicted output. Specifically, x packs the natural language query and index/schema; y denotes the desired (code, result) pair in the ES query and its execution. The task can be formally defined as:

$$\hat{y} = \arg \max_{y' \in \mathcal{Y}} S(y', y) \quad \text{s.t.} \quad y' \in \mathcal{V}(x).$$

Here, $\mathcal{Y}$ is the output space (valid Elasticsearch Queries), S is a scoring function defined over the predicted output and the target (e.g., Executable Code Ratio and Pass Rate used in Section 6), and $\mathcal{V}(x)$ enforces that outputs are syntactically typed and executable under the schema implied by x .

4 MedESQ Dataset Construction

In this section, we provide the detailed steps involved in constructing the **MedESQ** dataset for the text-to-ESQ task. Prior studies on NLQ have primarily adopted two approaches for complexity categorization. The Spider benchmark mainly relies on coarse measures such as the number of involved tables and basic SQL operations (Yu et al., 2018), without providing fine-grained distinctions. In contrast, BIRD categorizes queries using SQL structural classes and human reasoning difficulty (Li et al., 2023). However, these criteria remain insufficiently precise for broad and systematic use. Limited diversity in the time-complexity of text-query pairs often draws from similar schemas or with

Level	Function Composition Examples
Easy	id + term must + should filter + term + constant_score
Medium	prefix + term range + id regexp + prefix + id
Hard	match_phrase + sort multi_match + wildcard multi_match + match_phrase + sort

Table 3: Selected composed functions samples in different query categories based on the time complexity.

comparable levels of complexity. This drawback not only biases model training but also hinders fair performance comparisons (Jain et al., 2020). Therefore, in this paper, we introduce query time complexity as a key dimension in our **MedESQ** data generation process and group seven classes of Elasticsearch² queries (Section 3) into three primary complexity levels, accounting for the effect of data volume. Compared to seven purely mathematical classes, the three-based grouping is more suitable for practical applications. In the remaining of this section, we will introduce the details of the **MedESQ** dataset generation. The dataset and code will be publicly released after acceptance.

4.1 Source Data for Knowledge Screening

Specifically, our **MedESQ** Dataset is built from the publicly available VAERS³ dataset, which includes patient demographics, diagnoses, a standardized symptom vocabulary, and contextual information related to vaccine adverse events. VAERS is designed to detect potential vaccine safety signals and to support further evaluations.

4.2 MedESQ Dataset Generation

The **MedESQ** dataset was generated by following four primary steps. Figure 2 shows an illustrative example of the data generation.

4.2.1 Basic Function Collection (Step A)

We begin by systematically collecting all available functions from the Elasticsearch documentation. From this collection, we filter and retain only those functions that are relevant to text search operations for knowledge screening from VAERS data.

²<https://www.elastic.co/elasticsearch>

³<https://vaers.hhs.gov/>

4.2.2 Function Composition (Step B)

In this process, we first generate candidate operation pairs by combining the retained Elasticsearch functions in Step A. A secondary filtering stage is then applied to remove infeasible constructs, including syntactically invalid combinations and semantically unusual pairings (e.g., aggregations embedded within unsupported filter contexts). After producing the valid set of combinations, we organize them according to the query taxonomy introduced in Section 3.3. From this structured space, we select 300 commonly used function combinations, ensuring a uniform distribution across the three defined complexity levels.

4.2.3 ES Query Generation (Step C)

Using these function combinations, we then incorporated real values extracted from VAERS dataset. For each function combination, we select 30 distinct value pairs drawn from different table columns from VAERS (e.g., AGE_YRS, STATE, VAX_TYPE, and SYMPTOM_TEXT), ensuring diversity in both categorical and numeric attributes.

4.2.4 Natural Language Question Generation (Step D)

In the final step, the natural language question corresponding to each ES query is generated by GPT4o and further checked by human annotators. After an initial review, we identified some errors among the 9,000 text–query pairs that require further validation and correction. To address these issues, our experts conducted a comprehensive review of all pairs by considering the following three key aspects. First, **Function-combination validity**: We verified that each combination made semantic sense (e.g., the aggs function applies when needed group calculation). Second, **Query executability**: We ran query on our Elasticsearch server to ensure it returned results without errors. Third, **Question appropriateness**: We confirmed that each question was contextually sensible.

4.3 Basic Statistics of MedESQ Dataset

Figure 3 shows the distribution of the top three frequent function combinations of each complexity level in the MedESQ dataset. Upon completing data preparation, we had a clean dataset of over 6,000 NL-ESQ pairs, including 1,887 in the easy category, 1,737 in the medium category, and 2,779 in the hard category. The final dataset encompasses a wide range of query complexities, includ-

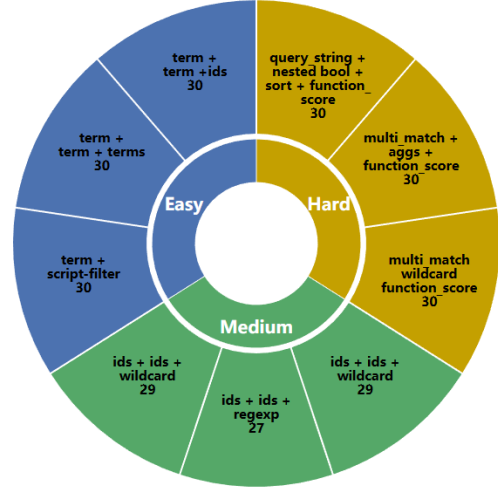


Figure 3: Distribution of the top three frequent function combinations in different difficulty levels in MedESQ.

ing single-level lookup, logical filtering, full-text search, and multi-level aggregation etc. In critical safety domains such as healthcare, the MedESQ dataset provides a solid foundation for evaluating the text-to-ESQ task through executable queries with comprehensive coverage of query complexity.

5 The RACE-ESQ Pipeline

To solve the text-to-ESQ task and benchmark on the MedESQ dataset, we guide LLMs to generate efficient ES queries from natural language questions by adopting the Retrieval-Augmented Generation (RAG) framework, which integrates both *reasoning* and *retrieval* to explicitly incorporate query complexity into the generation process. The RAG framework is particularly well-suited to the efficiency-oriented text-to-ESQ task, as ES query generation requires semantic understanding of natural language questions, accurate mapping to sequences of query operations, and external knowledge to guide efficiency-aware decisions. By explicitly separating reasoning from tool use, RAG enables the model to reason over query intent, conditions, constraints, and interact with external resources to validate or refine intermediate steps.

Based on the RAG framework, we designed a two-stage retrieval and reasoning-augmented framework for complexity-aware text-to-ESQ to ensure reliable query generation. Each prompt begins with the database schema to explicitly encode data structure and ensure consistent context across all baselines. The prompt body follows a two-stage process. First, an information retrieval

Table 4: Performance of six models on **MedESQ** dataset. Each score is the median of three runs on 200 sampled queries per level. The best performance is shown in **bold**, and the second best is underlined.

Model	Easy		Medium		Hard	
	ECR	PR	ECR	PR	ECR	PR
GPT4	80%	64%	77%	36%	55%	35%
GPT4o	85%	77%	<u>80%</u>	65%	67%	55%
GPT5	97%	87%	<u>80%</u>	67%	<u>75%</u>	63%
Grok4	97%	90%	72%	69%	71%	61%
GLM4.5	90%	75%	76%	62%	67%	40%
Qwen3	87%	80%	78%	60%	65%	53%
RACE-ESQ	<u>93%</u>	<u>87%</u>	82%	79%	79%	77%

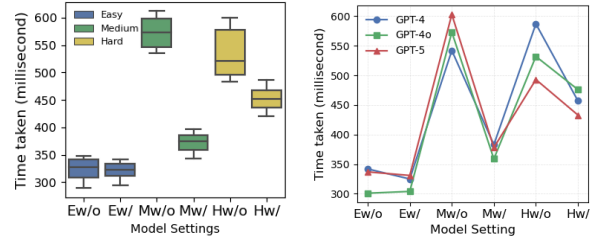
stage aligns relevant attributes with standardized schema fields, generates all possible operation strategies and chooses the lowest-cost one based on the dataset. Second, the retrieved information is transformed into executable queries, improving robustness across varying query complexity. Guided by a system prompt and a reasoning-action step, the model grounds decisions in retrieved evidence, reducing invalid function usage. Domain-specific Golden Rules further constrain reasoning, mitigate hallucinations, and ensure schema-consistent query generation in healthcare settings. The detailed prompts are provided in the Appendix.

6 Experiments

To evaluate the proposed benchmark and pipeline, we investigate three research questions (RQs) through a comprehensive experimental analysis. (1) **RQ1**: What is the performance of **RACE-ESQ** on the **MedESQ** dataset across each query difficulty level? (2) **RQ2**: How does the computational cost of the proposed **RACE-ESQ** method compare to original models on the **MedESQ** dataset? (3) **RQ3**: How should we interpret the outputs produced at different difficulty levels, and what relationships exist between those results?

6.1 Experimental Settings

We considered several proprietary large language models as baselines, including GPT4, GPT4o, and GPT5, accessed via the OpenAI API as black-box systems and other advanced models through their website: Grok4, GLM4.5 (Zeng et al., 2025), and Qwen3 (Yang et al., 2025), on the **MedESQ** dataset, followed by a comparative analysis. All models are evaluated in both zero-shot without fine-tuned settings. We set the temperature to 0.2 and



(a) Query runtime of GPT5 with/without **RACE-ESQ** (b) Query runtime from models with/without **RACE-ESQ**

Figure 4: Query execution time with and without applying **RACE-ESQ** across difficulty levels. Abbreviations indicate task difficulty and pipeline status: E_w and $E_{w/o}$ denote Easy (with / without **RACE-ESQ**). The same convention applies to medium and hard levels.

the maximum tokens to 600 for all models.

To assess the performance of different models, we employ two key metrics. (1) *Executable Code Ratio* (ECR): the proportion of generated code that can be successfully executed, (2) *Pass Rate* (PR): the ratio of generated code that produces correct results according to predefined test cases. Variations in the ECR across difficulty levels reflect the ability to organize and generate queries under increasing logical complexity, while changes in the PR indicate the model’s capacity to capture deeper semantic information required for correctness.

6.2 Experimental Results

6.2.1 Overall Performance on MedESQ (RQ1)

We evaluate all baselines and select GPT5 as the backbone of **RACE-ESQ** pipeline, as it achieves the best performance across the majority of settings shown in Table 4. Moreover, GPT5 is further improved when integrated into the pipeline. At the Easy level, all models perform well, though GPT4 lags in PR, indicating weaker information extraction. GPT5 shows clear gains within the GPT family, while Grok4 achieves the strongest Easy-level performance overall. GLM attains a competitive ECR score but exhibits a lower PR, revealing a gap between executability and semantic correctness.

At the Medium and Hard levels, PR declines across models as logical complexity increases, with GPT4 showing the largest drop. **RACE-ESQ** maintains the highest PR and a small ECR–PR gap, indicating greater stability. Moreover, it avoids inefficient operations, for example, it selects the prefix operator instead of GPT5’s default wildcard for prefix-based symptom queries, improving efficiency without sacrificing accuracy.

Table 5: Representative query generation errors across three difficulty levels in MedESQ. Each row shows the question pattern, expected gold query, common errors, and models that typically exhibit those errors.

Level	Question Pattern	Gold Query (Expected)	Common Errors	Models Impacted
Easy	Count by STATE for vaccine type = COVID19	Filter VAX_TYPE=COVID19, year=2023; aggs by STATE	Use VAX_NAME instead of VAX_TYPE; missing aggs	Qwen, GLM4.5, Grok4
	Count ER visits with L_THREAT=Y	Match L_THREAT:Y and ER_VISIT:Y	Drop one boolean constraint	GLM4.5, Qwen; rare in GPT4/4o
Medium	Avg hospital days by state for COVID19	Filter on VAX_TYPE, terms by STATE with avg(HOSP_DAYS)	Use sum instead of avg; wrong grouping field	GPT4 (avg/sum swap), GLM4.5, Grok4
	Monthly counts with prior vaccine and not died	Exists(PRIOR_VAX), DIED:N, date histogram (month)	Miss existence check; wrong bucket granularity	Qwen (existence), GLM4.5 (granularity)
	Top 5 states for VAX_MANU=Moderna	Filter on VAX_MANU, terms by STATE, size=5	Omit size cap; wrong sort	Grok4, GLM4.5
Hard	Must asthma + ER visit, should fever, top 10 states	bool.must(asthma, ER=Y); bool.should(fever); terms(STATE), size=10	Swap must/should; lose soft preference; wrong aggs order	GPT4/4o (swap), Qwen/GLM (lost preference), Grok4
	Conditional group by type vs. manufacturer	If AGE<12 → VAX_TYPE; else VAX_MANU; year=2023; size=5	Ignore branch; group by both; omit filter or size	GLM4.5, Qwen (branch collapse), Grok4, GPT5
	Rate of hospitalizations per state	Nested terms(STATE) with metric sum(HOSPITAL=Y)/count(all)	Raw counts only; missing denominator normalization	Qwen, GLM4.5, Grok4; GPT4/4o/5

6.2.2 Computational Cost Analysis (RQ2)

We evaluate the efficiency improvement of our pipeline by randomly sampling 10 queries per difficulty level generated by GPT family in Figure 4(b), and comparing the total query execution time with and without our method. In addition, we run every experiment 20 times to illustrate variability. All experiments were conducted on a single-cluster ES database of approximately 100 MB, containing 50,000 records. Figure 4(a) reports the results of GPT5. Across this process, our pipeline yields consistent latency reductions, particularly for Medium and Hard level queries, which are known to change the original operation to a more highly efficient operator. For example, the runtime of GPT5 queries at the Medium level are reduced from 603 ms to 378 ms, while Hard queries decrease from 493 ms to 433 ms, with Medium-level reductions exceeding 35% in some cases. In contrast, Easy queries show marginal changes, reflecting that simple exact-match queries already operate near optimal cost and offer limited room for optimization. Overall, the highly variable execution time observed in the original models highlights a clear gap in current research on different time-complexity query generation.

6.2.3 Error Analysis (RQ3)

Although the results highlight the potential of NLP methods for the text-to-ESQ task, a number of errors still occur during query generation. We analyze model behaviors on the MedESQ across three difficulty levels and identify recurrent failure types with practical examples in Table 5. Errors on the easy level mainly stem from entity misclassification and schema drift, such as confusing VAX_TYPE with VAX_NAME or omitting simple groupings. For instance, when filtering by “VAX_TYPE = COVID19” and aggregating by STATE, some models instead match brand names (e.g., PFIZER-BIONTECH) or drop the grouping clause. GPT5 shows the strongest schema fidelity, while GPT4/4o occasionally over-normalizes values. Qwen3 and GLM4.5 sometimes omit constraints in long prompts, and Grok4 struggles with brand aliases.

Medium-level errors involve resolving implicit attributes and aggregations, such as mapping “teenager” to AGE_YRS ∈ [13, 19] or choosing count vs. avg. GPT5 handles these reliably, while others misselect operators or omit aggs.

Hard-level errors arise from compositional logic and operator orchestration, including misusing bool.must vs. bool.should. GPT5 remains most stable, whereas other models misorder filters, flatten nested aggs, or simplify multi-branch logic.

7 Conclusion

Most existing work on efficiency-oriented query generation focus on text-to-SQL tasks and lacks a fundamental analysis of efficiency factors, which causes heavily rely on the capabilities of LLMs. In this paper, we conduct a complexity protocol for NoSQL, in particular, Elasticsearch. Following, We construct the complexity-aware Medical text-to-ElasticSearch Queries (**MedESQ**) dataset based on VAERS data and develop a two-stage RAG framework, **Retrieval-Augmented** pipeline for **Complexity-aware** text-to-ESQ generation (**RACE-ESQ**) for evaluating the new **MedESQ** data. Experimental results demonstrate that our approach generates highly efficient and high-quality Elasticsearch queries.

Limitations

This work focuses exclusively on Elasticsearch, and its design is tightly coupled with Lucene-based index structures. In addition, the RAG pipeline relies on fixed schema mappings, which may constrain the expressive capacity of large language models. Alternative approaches, such as model fine-tuning, could further improve performance given sufficient computational resources. Finally, while **MedESQ** covers diverse query types, large-scale experiments on substantially larger datasets would be better to fully assess scalability and real-world robustness. However, the efficiency-oriented design of the **MedESQ** dataset represents the first comprehensive investigation of this problem space and provides a benchmark resource for the research community to support the design and evaluation of future methods.

Ethical Considerations

This work uses publicly available de-identified VAERS data and does not involve human subjects.

References

Andrzej Bialecki, Robert Muir, Grant Ingersoll, and Lucid Imagination. 2012. Apache lucene 4. In *SIGIR 2012 workshop on open source information retrieval*, page 17.

Naihao Deng, Yulong Chen, and Yue Zhang. 2022. Recent advances in text-to-sql: A survey of what we have and what we expect. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 2166–2187.

Xiang Deng, Ahmed Hassan, Christopher Meek, Oleksandr Polozov, Huan Sun, and Matthew Richardson. 2021. Structure-grounded pretraining for text-to-sql. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 1337–1350.

Kazi Mojammel Hossen, Mohammed Nasir Uddin, Minhazul Arefin, and Md Ashraf Uddin. 2023. Bert model-based natural language to nosql query conversion using deep learning approach. *International Journal of Advanced Computer Science and Applications*, 14(2).

Abhinav Jain, Hima Patel, Lokesh Nagalapatti, Nitin Gupta, Sameep Mehta, Shanmukha Guttula, Shashank Mujumdar, Shazia Afzal, Ruhi Sharma Mittal, and Vitobha Munigala. 2020. Overview and importance of data quality for machine learning tasks. In *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, pages 3561–3562.

Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, and 1 others. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows. *The Thirteenth International Conference on Learning Representations*.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, and 1 others. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474.

Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, and 1 others. 2023. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Advances in Neural Information Processing Systems*, 36:42330–42357.

Jinwei Lu, Yuanfeng Song, Zhiqian Qin, Haodi Zhang, Chen Zhang, and Raymond Chi-Wing Wong. 2025. Bridging the gap: Enabling natural language queries for nosql databases through text-to-nosql translation. In *arXiv preprint arXiv:2502.11201*.

Mohammadreza Pourreza, Ruoxi Sun, Hailong Li, Lesly Miculicich, Tomas Pfister, and Sercan Ö. Arik. 2025. Sql-gen: Bridging the dialect gap for text-to-sql via synthetic data and model merging. In *International Conference on Learning Representations (ICLR)*.

Zhiqian Qin, Yuanfeng Song, Jinwei Lu, Yuanwei Song, Shuaimin Li, and Chen Jason Zhang. 2025. Multitend: A multilingual benchmark for natural language to nosql query translation. In *arXiv preprint arXiv:2502.11022*.

- Vivek Basavegowda Ramu. 2023. Optimizing database performance: Strategies for efficient query execution and resource utilization. *International Journal of Computer Trends and Technology*, 71(7):15–21.
- Jie Shi, Bo Xu, Jiaqing Liang, Yanghua Xiao, Jia Chen, Chenhao Xie, Peng Wang, and Wei Wang. 2025. Gen-sql: Efficient text-to-sql by bridging natural language question and database schema with pseudo-schema. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 3794–3807.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, and 1 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, and 1 others. 2018. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*.
- Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, and 1 others. 2025. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*.
- Wenlong Zhang, Tian Shi, and Ping Wang. 2024. Natural language querying on nosql databases: Opportunities and challenges [vision paper]. In *2024 IEEE International Conference on Big Data (BigData)*, pages 3353–3357. IEEE.
- Wenlong Zhang, Kangping Zeng, Xinming Yang, Tian Shi, and Ping Wang. 2023. Text-to-esq: A two-stage controllable approach for efficient retrieval of vaccine adverse events from nosql database. In *Proceedings of the 14th ACM International Conference on Bioinformatics, Computational Biology, and Health Informatics*, pages 1–10.
- Yizhang Zhu, Runzhi Jiang, Boyan Li, Nan Tang, and Yuyu Luo. 2025. Elliesql: Cost-efficient text-to-sql with complexity-aware routing. *arXiv preprint arXiv:2503.22402*.

A RACE-ESQ Prompt Framework

To improve both execution correctness and query efficiency, RACE-ESQ uses a two-stage retrieval-augmented framework. The first stage grounds the user question in the database schema and retrieves candidate operator patterns from the benchmark. The second stage composes the final Elasticsearch query under a set of cost-aware constraints. This appendix reorganizes the method description around *inputs*, *retrieval*, *generation*, and *evaluation details*, so that implementation choices are easier to follow.

A.1 Input Representation

Each prompt contains three components:

1. the natural-language question;
2. the database schema, including the available VAERS fields;
3. optional normalized medical terms or operator candidates retrieved in Stage 1.

The model returns a structured object with the following fields:

- `standard_terms`: normalized symptoms or value strings used for retrieval;
- `retrieved_patterns`: candidate query templates or operator plans;
- `es_query`: the final executable Elasticsearch query.

A.2 Stage 1: Schema Grounding and Pattern Retrieval

Stage 1 identifies the fields, values, and operator families implied by the question.

1. **Entity and symptom grounding.** The model extracts symptom phrases, demographic conditions, temporal constraints, and aggregation intents from the question.
2. **Schema linking.** Each extracted span is aligned to a valid field in the Elasticsearch schema, such as `SYMPTOM_TEXT`, `VAX_TYPE`, `STATE`, or `AGE_YRS`.
3. **Pattern retrieval.** The system retrieves candidate operator patterns from MedESQ that are compatible with the grounded intent.
4. **Complexity-aware ranking.** Among valid candidates, RACE-ESQ prefers lower-cost patterns when they satisfy the same semantics.

A.3 Stage 2: Cost-Aware Query Composition

Given the grounded fields and retrieved patterns, Stage 2 synthesizes the final Elasticsearch query. The generator is encouraged to:

- prefer term, range, and prefix over more expensive operators when the intent allows;
- use full-text operators only for free-text fields such as `SYMPTOM_TEXT`;
- preserve schema compliance and output executable JSON;
- produce a minimal query when the user intent is ambiguous.

A.4 System Prompt

The Elasticsearch database contains the following fields: `RECVDATE`, `STATE`, `AGE_YRS`, `VAERS_ID`, `SEX`, `SYMPTOM_TEXT`, `DIED`, `ER_VISIT`, `L_THREAT`, `HOSPITAL`, `HOSPDAYS`, `DISABLE`, `VAX_DATE`, `LAB_DATA`, `OTHER_MEDS`, `CUR_ILL`, `HISTORY`, `PRIOR_VAX`, `TODAYS_DATE`, `OFC_VISIT`, `VAX_TYPE`, `VAX_MANU`, `VAX_LOT`, `VAX_DOSE_SERIES`, `VAX_NAME`, `ALLERGIES`.

You are an expert Elasticsearch engineer. Convert each natural-language question into a correct, minimal, and schema-compliant Elasticsearch query. Prefer lower-cost operators whenever they preserve the intended semantics. Do not introduce fields that are not present in the schema.

A.5 Golden Rules

1. Use only fields that appear in the schema.
2. Normalize symptom expressions before generating the final query.
3. Prefer exact filters (term, range, prefix) for structured fields.
4. Use `match`, `match_phrase`, or `multi_match` only for free-text search.
5. When multiple valid plans exist, prefer the lower-cost pattern.
6. If the question is underspecified, return a safe minimal query rather than hallucinating extra constraints.
7. Output the intermediate grounding fields (`standard_terms` and `retrieved_patterns`) together with the final `es_query` for analysis.

A.6 Worked Example

Input question. *Show reports that mention chest pain after a COVID-19 vaccine for patients aged 18 or older.*

Stage 1 output.

- standard_terms: ["chest pain", "COVID19"]
- retrieved_patterns: ["bool.must", "term", "range", "match_phrase"]
- grounded fields: SYMPTOM_TEXT, VAX_TYPE, AGE_YRS

Stage 2 output.

```
{
  "standard_terms": ["chest pain", "COVID19"],
  "retrieved_patterns": ["bool.must", "term", "range", "match_phrase"],
  "es_query": {
    "query": {
      "bool": {
        "must": [
          {"match_phrase": {"SYMPTOM_TEXT": "chest pain"}},
          {"term": {"VAX_TYPE": {"value": "COVID19"}}},
          {"range": {"AGE_YRS": {"gte": 18}}}
        ]
      }
    }
  }
}
```

B Representative Query Templates by Complexity Class

This section provides short, syntax-correct examples for each complexity class discussed in Section 3. The examples are illustrative and may require field-specific mappings (for example, keyword subfields) depending on the index configuration.

B.1 Class A: Constant Lookup

```
GET vaers/_doc/123456
```

B.2 Class B: Exact Term Query

```
GET vaers/_search
{
  "query": {
    "term": {
      "ER_VISIT": { "value": "Y" }
    }
  }
}
```

B.3 Class C: Boolean Composition

```
GET vaers/_search
{
  "query": {
    "bool": {
      "filter": [
```

```
    { "term": { "DISABLE": { "value": "Y" } } },
    { "term": { "L_THREAT": { "value": "Y" } } }
  ]
}
}
```

B.4 Class D: Numeric Range

```
GET vaers/_search
{
  "query": {
    "range": {
      "AGE_YRS": { "gte": 18, "lte": 65 }
    }
  }
}
```

B.5 Class D: Prefix

```
GET vaers/_search
{
  "query": {
    "prefix": {
      "VAX_TYPE": { "value": "COVID" }
    }
  }
}
```

B.6 Class E: Wildcard

```
GET vaers/_search
{
  "query": {
    "wildcard": {
      "VAX_NAME": { "value": "*PFIZER*" }
    }
  }
}
```

B.7 Class F/G: Full-Text Search with Sorting or Aggregation

```
GET vaers/_search
{
  "size": 10,
  "query": {
    "match_phrase": {
      "SYMPTOM_TEXT": "chest pain"
    }
  },
  "sort": [
    { "RECVDATE": "desc" }
  ]
}
```

C Dataset Construction and Quality Control

MedESQ is built from the publicly available VAERS dataset and organized around the complexity taxonomy introduced in the main paper. The construction pipeline consists of four steps.

C.1 Step A: Operator Inventory

We first collect Elasticsearch functions that are relevant to healthcare search and safety surveillance.

The retained set includes lookup, boolean, range, pattern, full-text, aggregation, and sorting operators.

C.2 Step B: Valid Function Composition

We compose candidate operator patterns and remove invalid combinations. Filtering is applied at two levels:

- **syntactic validity:** the composed query must be executable JSON;
- **semantic validity:** the operator combination must be meaningful for the intended question type.

C.3 Step C: Value Instantiation

Each valid pattern is instantiated with real values sampled from VAERS fields such as AGE_YRS, STATE, VAX_TYPE, VAX_MANU, and SYMPTOM_TEXT. This step increases attribute diversity and reduces overfitting to a narrow set of lexical templates.

C.4 Step D: Natural-Language Question Generation

Natural-language questions are generated from the instantiated queries and then reviewed by human annotators. The review protocol checks:

1. **function-combination validity;**
2. **query executability;**
3. **question appropriateness and readability.**

After filtering and review, the final dataset contains more than 6,000 executable text–ESQ pairs distributed across easy, medium, and hard difficulty levels.

D Additional Experimental Details

D.1 Model Settings

All models are evaluated in zero-shot settings. The main paper reports results for GPT-4, GPT-4o, GPT-5, Grok4, GLM-4.5, Qwen3, and the RACE-ESQ pipeline built on top of GPT-5. Temperature is set to 0.2 and the maximum output length is set to 600 tokens.

D.2 Evaluation Metrics

We report two main metrics:

- **Executable Code Ratio (ECR):** the percentage of generated queries that execute without syntax or schema errors;

- **Pass Rate (PR):** the percentage of generated queries that return the correct answer under the predefined test cases.

To analyze efficiency, we additionally measure query latency on the Elasticsearch server.

D.3 Runtime Measurement Protocol

For latency experiments, each query is executed multiple times on the same Elasticsearch instance and the results are aggregated across repeated runs. To strengthen reproducibility, a final version of the paper could additionally report:

- median latency together with the interquartile range;
- warm-cache and cold-cache settings separately;
- the index size, hardware profile, and shard configuration used for each run.

D.4 Error Categories

The qualitative analysis can be organized into the following failure categories:

1. schema-linking errors;
2. value-normalization errors;
3. operator-selection errors;
4. aggregation and grouping errors;
5. compositional reasoning errors;
6. safe-minimal-query failures under ambiguity.

E Potential Additional Experiments and Analyses

Table 6 summarizes additional experiments that could strengthen the appendix or a future version of the paper.

E.1 Recommended Priority Order

If space is limited, the highest-value additions are:

1. a component ablation of RACE-ESQ;
2. a scaling study across larger index sizes;
3. a paraphrase-robustness evaluation;
4. a more fine-grained operator-level error analysis.

These experiments would directly support the paper’s core claim that explicit complexity awareness improves both generation quality and execution efficiency.

Study	Question	Main metrics	Expected insight
Component ablation	How much does each stage contribute? Compare the full system against variants without schema grounding, pattern retrieval, cost-aware selection, or golden rules.	ECR, PR, latency	Isolates which components are responsible for accuracy gains and which ones drive efficiency improvements.
Scaling by index size	How stable are the complexity tiers when the index grows from small to large collections?	Latency, PR	Tests whether the proposed taxonomy remains predictive under larger data volumes.
Paraphrase robustness	Do paraphrased questions preserve the same query plan and execution result?	ECR, PR	Measures robustness to linguistic variation beyond surface templates.
Unseen composition generalization	Can the model handle operator combinations that are held out during training or retrieval?	ECR, PR	Evaluates compositional generalization, which is critical for real deployment.
Cross-backend transfer	How well do the retrieved plans transfer to OpenSearch or other Lucene-style backends?	Executability, latency	Clarifies which findings are Elasticsearch-specific and which transfer across related engines.
Cost-quality frontier	How do token budget, reasoning depth, and latency trade off against execution quality?	PR, total inference time	Helps position RACE-ESQ as a practical system rather than only a benchmark method.
Human usefulness study	Are the generated queries understandable and acceptable to domain users?	Human ratings	Adds an interpretability and usability perspective that complements automatic metrics.

Table 6: Potential additional experiments that could strengthen the empirical section.